



ITES
RENÉ DESCARTES

Carrera:
Programación y WebMaster

Docente:
Mario Alonso Segovia Gutiérrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Inurreta Brito Gustavo Enrique
Carlos Alberto Castillo Pinzon
Rodrigo Alejandro Barbosa Vidal
Michael Roman Alavez Colli

Sistema de Gestión de Inventario y Resguardo de Bienes Institucionales

Sistema de Gestión de Inventario y Resguardo de Bienes Institucionales	2
Introducción	4
Propósito del documento	5
Alcance del sistema	5
Objetivos del proyecto	5
Descripción general del sistema	6
Perspectiva general del sistema	6
Funcionalidades principales	6
Tipos de usuarios	6
Restricciones generales	6
Dependencias externas	7
Stakeholders y actores	7
1.Casos de uso - Alta de bien	10
2.Casos de uso - Baja de bien	11
3.Casos de uso - Consulta de bien por código de barras	11
4.Casos de uso - Asignación de bienes a personal	11
5.Casos de uso - Reasignación de bienes	12
6.Casos de uso - Alta de personal	12
7.Casos de uso - Baja de personal	12
8.Casos de uso - Consulta de inventario	13
9.Casos de uso - Generación de reportes	13
2. METODOLOGÍA DE DESARROLLO SELECCIONADA	15
Sprint 5 – Vista de Visualizador	19
Herramienta utilizada	21
Repositorio del proyecto	21
Estrategia de ramas	21
Procedimiento para la integración de cambios	21
1. Reglas de Clean Code del Equipo	24
1. Generalidades del Proyecto	29
Objetivo General	29
Problema que Resuelve	29
Tecnologías Utilizadas	29
Arquitectura General del Sistema	30
Módulos Principales	30
2. Estrategia de Pruebas	31
Tipos de Pruebas a Realizar	31
Momento de Ejecución	31

Responsables	32
• Rodrigo Alejandro Barbosa Vidal	32
Herramientas para Documentar Resultados	32
3. Plan de Pruebas Unitarias	33
PU-01: Autenticación	33
PU-02: Registro de Bienes	33
PU-03: Importación Masiva	34
PU-04: Gestión de Asignaciones	36
PU-05: Resolución de Pendientes	37
PU-06: Exportación de Reportes	38
PU-07: Consulta Pública	39
4. Plan de Pruebas de Integración	40
PI-01: Acceso Seguro al Dashboard	40
PI-02: Integración de Catálogos en Bienes	40
PI-03: Trazabilidad de Asignaciones	40
PI-04: Consistencia en Importación Masiva	41
PI-05: Integridad de Datos Filtrados en Reportes	41
PI-06: Generación y Lectura de QR	42
5. Plan de Pruebas End-to-End (E2E)	42
E2E-01: Flujo Completo de Registro de un Bien	42
E2E-02: Ciclo de Asignación y Auditoría	43
E2E-03: Auditoría y Exportación de Reporte Anual	44
E2E-04: Carga de Inventario por Lote (Migración)	44
E2E-05: Escaneo y Consulta Externa (QR)	45
7. Criterios de Aceptación	48
Conclusión	51

Introducción

En este documento se presenta la planeación del proyecto **Sistema de Gestión de Inventario y Resguardo de Bienes Institucionales**, el cual tiene como objetivo ayudar a mejorar la forma en que se lleva el control de los bienes de la institución. Actualmente, parte de esta información se maneja mediante archivos de Excel y anotaciones, lo que puede provocar datos duplicados, errores, pérdida de información y dificultades para saber quién tiene asignado cada bien.

A lo largo del documento se explica cómo estará organizado el sistema, qué funciones tendrá, quiénes podrán utilizarlo y qué reglas se deberán seguir. Entre sus principales funciones se encuentran registrar, modificar y dar de baja bienes, asignarlos a una persona o área, realizar reasignaciones, consultar el inventario, utilizar filtros y generar reportes.

También se presenta la forma en que el equipo planea trabajar durante el desarrollo del proyecto, la organización de las actividades, las reglas que se seguirán para mantener un trabajo ordenado y las pruebas que se realizarán para comprobar que el sistema funcione correctamente. Con esta información se busca tener una idea clara de lo que se quiere desarrollar y contar con una guía que ayude al equipo durante todo el proyecto.

Propósito del documento

El propósito de este documento es presentar de forma clara los requerimientos del sistema de inventario, explicando sus funciones principales, usuarios, reglas de negocio y el alcance general del proyecto.

Alcance del sistema

El sistema permitirá administrar bienes institucionales, registrar altas y bajas, generar códigos de barras, consultar información de bienes, asignarlos a personal, reasignarlos, gestionar áreas y generar reportes de inventario, resguardo, áreas, personal y bienes.

Objetivos del proyecto

- Desarrollar un sistema que facilite el control de inventario de bienes
- Reducir errores en el registro manual de información
- Control de bienes y asignaciones por persona o área
- Generar reportes sobre bienes y saber altas y bajas

Descripción general del problema

La información del inventario se maneja mediante archivos en Excel, hay datos no registrados, errores y datos duplicados, pérdida de datos e información . Además, la asignación de bienes a las áreas y personal es complicado, no existe un sistema que lo controle.

Descripción general del sistema

Perspectiva general del sistema

El sistema funcionará como una plataforma web para administrar el inventario , Nombre del bien, Marca, Modelo, Serie, ID-SEP y áreas.

Funcionalidades principales

- Alta, baja y modificación de bienes.
- Generación de códigos de barras.
- Consulta de bienes mediante código de barras.
- Asignación y reasignación de bienes.
- Alta y baja de personal.
- Alta y baja de áreas.
- Consulta de inventario general.
- Filtros por área, personal y estatus.
- Gestión de activos pendientes.
- Generación e impresión de reportes.

Tipos de usuarios

Administrador:

Puede registrar, modificar, dar de baja, asignar, reasignar y generar reportes.

Visualizador:

Solo puede consultar información, aplicar filtros y visualizar reportes.

Restricciones generales

- Solo el administrador podrá modificar información del inventario.
- No se podrá eliminar personal que tenga bienes asignados.
- No se podrá asignar un bien dado de baja.
- Cada bien deberá tener un código de barras único.

Dependencias externas

El sistema dependerá de una base de datos para almacenar la información.

Se podrá utilizar lector de código de barras para consultar bienes.

Stakeholders y actores

Stakeholders principales

- Personal administrativo.
- Responsable del inventario.
- Personal que recibe bienes en resguardo.
- Directivos o encargados de área.

- Usuarios visualizadores.

Actores del sistema

- Administrador.
- Visualizador.
- Personal asignado a bienes.
- Área administrativa.

Intereses y responsabilidades

El administrador será responsable de mantener actualizado el inventario.

El visualizador podrá revisar información sin modificar datos.

El personal asignado será responsable de los bienes que tenga en resguardo.

Los directivos podrán consultar reportes para conocer el estado del inventario del área.

Requerimientos funcionales

ID	Requerimiento funcional
RF-01	El sistema permitirá registrar nuevos bienes con datos como ID-SEP, No. Inventario, nombre del bien, marca, modelo, serie, ADQ, valor y resguardo actual.
RF-02	El sistema permitirá dar de baja bienes registrados en el inventario.
RF-03	El sistema generará un código de barras único para cada bien registrado.
RF-04	El sistema permitirá consultar bienes mediante el escaneo o ingreso del código de barras.
RF-05	El sistema permitirá asignar bienes a personal registrado.
RF-06	El sistema permitirá reasignar bienes de una persona a otra.
RF-07	El sistema permitirá registrar nuevo personal.
RF-08	El sistema permitirá dar de baja personal siempre que no tenga bienes asignados.
RF-09	El sistema bloqueará la baja de personal que tenga bienes bajo resguardo.
RF-10	El sistema permitirá registrar nuevas áreas.
RF-11	El sistema permitirá dar de baja áreas que no tengan bienes o personal asociados.
RF-12	El sistema permitirá asignar bienes a un área específica.
RF-13	El sistema permitirá consultar el inventario general.
RF-14	El sistema permitirá filtrar el inventario por área.
RF-15	El sistema permitirá filtrar el inventario por personal.
RF-16	El sistema permitirá filtrar el inventario por estatus.
RF-17	El sistema permitirá visualizar activos pendientes de asignación o revisión.
RF-18	El sistema generará reportes generales de bienes.
RF-19	El sistema generará reportes de resguardo.
RF-20	El sistema generará reportes por área.
RF-21	El sistema generará reportes por personal.
RF-22	El sistema generará reportes por tipo de bien.
RF-23	El sistema permitirá buscar bienes por nombre, número de inventario, serie o resguardo actual.
RF-24	El sistema permitirá imprimir reportes generados.
RF-25	El sistema permitirá al administrador modificar información de bienes registrados.
RF-26	El sistema permitirá al visualizador consultar información sin poder editarla.

Requerimientos no funcionales

ID	Requerimiento no funcional
RNF-01	El sistema deberá responder a las consultas de inventario en un tiempo máximo de 3 segundos.
RNF-02	El sistema deberá contar con inicio de sesión para controlar el acceso de usuarios.
RNF-03	El sistema deberá manejar roles de usuario: administrador y visualizador.
RNF-04	El sistema deberá estar disponible durante el horario laboral de la institución.
RNF-05	El sistema deberá permitir el crecimiento del inventario sin afectar su funcionamiento.
RNF-06	El sistema deberá tener una interfaz sencilla y fácil de usar.
RNF-07	El sistema deberá registrar acciones importantes como altas, bajas, modificaciones y reasignaciones.
RNF-08	El sistema deberá permitir respaldo periódico de la información.
RNF-09	El sistema deberá proteger la información contra modificaciones no autorizadas.
RNF-10	El sistema deberá permitir la impresión clara de los reportes generados.

Casos de uso

1.Casos de uso - Alta de bien

Objetivo: Registrar un nuevo bien en el inventario.

Actor principal: Administrador.

Precondiciones: El administrador debe haber iniciado sesión.

Flujo principal:

- El administrador entra al módulo de bienes.
- El administrador selecciona la opción de “Agregar bien”.
- Ingresa los datos del bien.
- El sistema valida y guarda la información.
- El sistema genera un código de barras al bien.

Flujos alternos:

Si falta un dato obligatorio, el sistema muestra un mensaje de error.

Postcondiciones:

El bien queda registrado en el inventario.

2.Casos de uso - Baja de bien

Objetivo: Dar de baja un bien del inventario.

Actor principal: Administrador.

Precondiciones: El bien debe existir en el sistema.

Flujo principal:

- El administrador busca el bien.
- Selecciona la opción “Dar de baja”.
- Captura el motivo de baja.
- El sistema cambia el estatus del bien.

Flujos alternos:

Si el bien no existe, el sistema muestra un mensaje.

Postcondiciones:

El bien queda marcado como dado de baja.

3.Casos de uso - Consulta de bien por código de barras

Objetivo: Consultar la información de un bien usando su código de barras.

Actor principal: Administrador o visualizador.

Precondiciones: El bien debe tener código de barras registrado.

Flujo principal:

- El usuario entra al módulo de consulta.
- Escanea o escribe el código de barras.
- El sistema busca el bien.
- El sistema muestra la información del bien.

Flujos alternos:

Si el código no existe, el sistema muestra que no se encontró información.

Postcondiciones:

El usuario visualiza los datos del bien.

4.Casos de uso - Asignación de bienes a personal

Objetivo: Asignar un bien a una persona responsable.

Actor principal: Administrador.

Precondiciones: El bien y el personal deben estar registrados.

Flujo principal:

- El administrador busca el bien.
- Selecciona la opción "Asignar".
- Elige al personal responsable.
- El sistema registra el resguardo.

Flujos alternos:

Si el bien está dado de baja, no podrá asignarse.

Postcondiciones:

El bien queda asignado al personal seleccionado.

5.Casos de uso - Reasignación de bienes

Objetivo: Cambiar el responsable de un bien.

Actor principal: Administrador.

Precondiciones: El bien debe estar asignado previamente.

Flujo principal:

- El administrador consulta el bien.
- Selecciona "Reasignar".
- Elige al nuevo responsable.
- El sistema actualiza el resguardo actual.

Flujos alternos:

Si el nuevo responsable no existe, el sistema solicitará registrarlo primero.

Postcondiciones:

El bien queda asignado al nuevo responsable.

6.Casos de uso - Alta de personal

Objetivo: Registrar personal dentro del sistema.

Actor principal: Administrador.

Precondiciones: El administrador debe iniciar sesión.

Flujo principal:

- El administrador entra al módulo de personal.
- Selecciona "Agregar personal".

- Captura los datos solicitados.
- El sistema valida y guarda la información.

Flujos alternos:

Si el personal ya existe, el sistema muestra una alerta.

Postcondiciones:

El personal queda disponible para asignación de bienes.

7.Casos de uso - Baja de personal

Objetivo: Dar de baja personal registrado.

Actor principal: Administrador.

Precondiciones: El personal debe existir en el sistema.

Flujo principal:

- El administrador busca al personal.
- Selecciona "Dar de baja".
- El sistema verifica si tiene bienes asignados.
- Si no tiene bienes, se realiza la baja.

Flujos alternos:

Si tiene bienes asignados, el sistema bloquea la baja.

Postcondiciones:

El personal queda dado de baja si cumple las condiciones.

8.Casos de uso - Consulta de inventario

Objetivo: Visualizar los bienes registrados.

Actor principal: Administrador o visualizador.

Precondiciones: El usuario debe iniciar sesión.

Flujo principal:

- El usuario entra al módulo de inventario.
- El sistema muestra la lista de bienes.
- El usuario puede aplicar filtros por área, personal o estatus.

Flujos alternos:

Si no existen bienes registrados, el sistema muestra inventario vacío.

Postcondiciones:

El usuario visualiza el inventario.

9.Casos de uso - Generación de reportes

Objetivo: Generar reportes del inventario.

Actor principal: Administrador o visualizador.

Precondiciones: Debe existir información registrada.

Flujo principal:

- El usuario entra al módulo de reportes.

- Selecciona el tipo de reporte.
- Aplica filtros si lo desea.
- El sistema genera el reporte.
- El usuario puede imprimirlo.

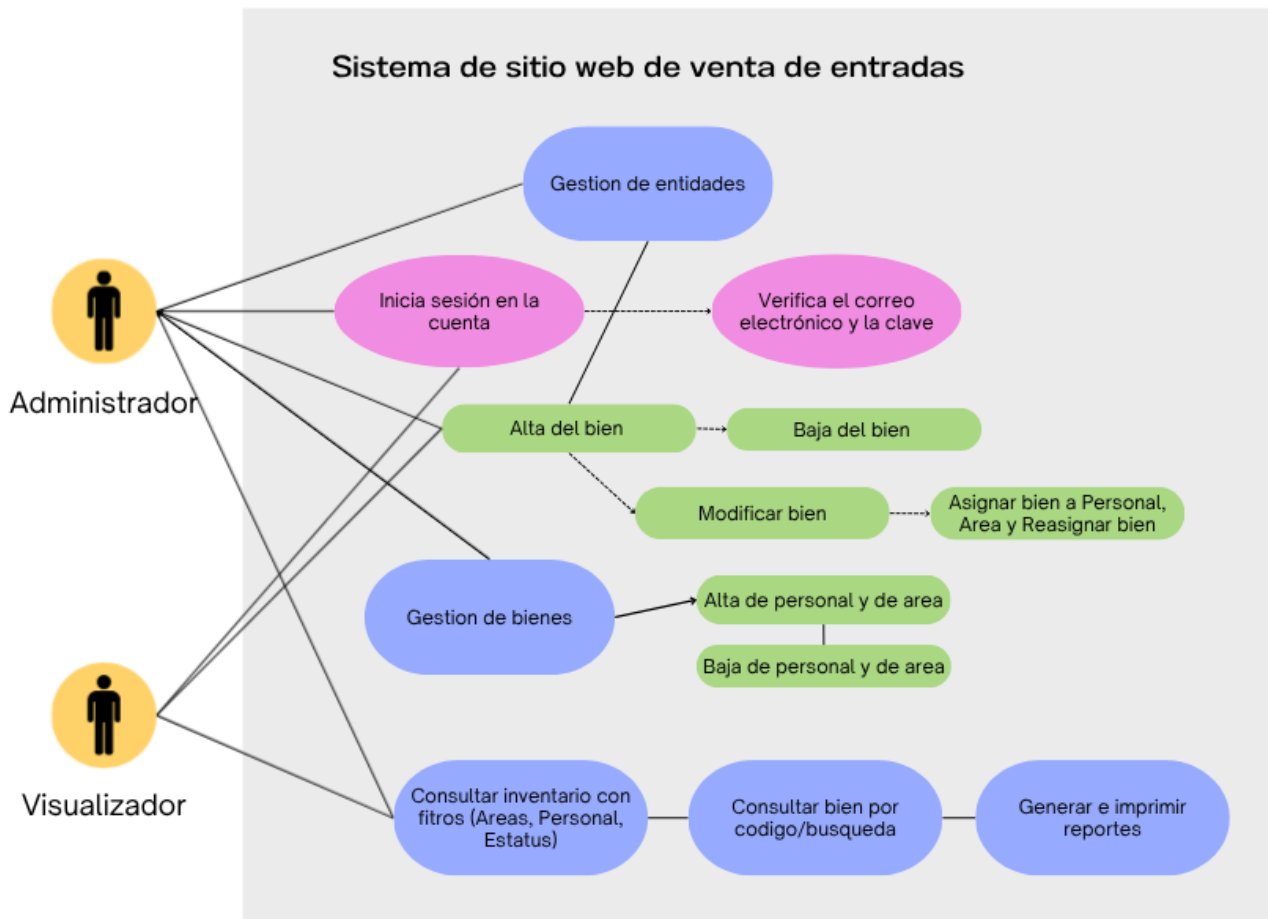
Flujos alternos:

Si no hay datos, el sistema muestra un mensaje.

Postcondiciones:

El reporte queda generado para consulta o impresión.

Diagrama de Casos de Uso



Reglas de negocio

ID	Regla de negocio
RN-01	Todo bien registrado deberá tener un número de inventario único.
RN-02	Cada bien deberá contar con un código de barras único.
RN-03	Un bien solo podrá estar asignado a una persona a la vez.
RN-04	Un bien dado de baja no podrá ser asignado a personal.
RN-05	El personal no podrá darse de baja si tiene bienes asignados.
RN-06	Solo el administrador podrá registrar, modificar o dar de baja información.
RN-07	El visualizador no podrá realizar cambios en el inventario.
RN-08	Toda reasignación deberá actualizar el resguardo actual del bien.
RN-09	Los reportes deberán generarse con información actualizada del sistema.
RN-10	Las áreas no podrán eliminarse si tienen bienes o personal asociados.
RN-11	El sistema deberá conservar historial de asignaciones y reasignaciones.
RN-12	Los bienes pendientes deberán aparecer en una sección especial para su revisión.

Información del proyecto

1. DESCRIPCIÓN GENERAL DEL PROYECTO

Nombre del proyecto:

Sistema de Gestión de Inventario y Resguardo de Bienes Institucionales

Cliente o usuario objetivo:

Personal administrativo del inventario institucional y dirección del plantel.

Integrantes del equipo:

- Gustavo Enrique Inurreta Brito
- Michael Román Alavez Collí
- Carlos Alberto Castillo Pinzon
- Rodrigo Alejandro Barbosa Vidal

Objetivo general del sistema

Desarrollar una plataforma para administrar y controlar bienes institucionales mediante el registro, consulta, asignación, reasignación y generación de reportes de inventario.

Problema que busca resolver

El inventario se administra mediante archivos de Excel, lo que provoca errores, información duplicada y dificultades para controlar la asignación de bienes. Además, algunos bienes siguen registrados a nombre de personal jubilado o que ya no labora en la institución, lo que genera información desactualizada y problemas en el control del inventario.

2. METODOLOGÍA DE DESARROLLO SELECCIONADA

Metodología elegida

La metodología seleccionada para el desarrollo del proyecto es Scrum, utilizando un tablero Kanban como apoyo para la organización y seguimiento de las tareas por medio de notion.

Justificación

¿Por qué seleccionaron esta metodología?

Se seleccionó la metodología scrum que permite organizar el desarrollo por medio de etapas y sprints dando seguimiento de las actividades y de la entrega progresiva de las partes funcionales de la plataforma y la manera de organizarnos

fue por medio de la metodología kanban como apoyo visual.

¿Qué ventajas ofrece para su proyecto?

La facilidad de adaptación de cambios durante el desarrollo del proyecto y mayor organización del trabajo manteniendo el control de los avances.

¿Qué características del sistema hacen que sea adecuada?

Los diferentes módulos que gestionan los bienes, personal, áreas, asignaciones y reportes lo cual se puede desarrollar por módulos los cuales se pueden desarrollar de una forma independiente y ya irse integrando por eso decidimos que la metodología Scrum es la adecuada.

¿Qué beneficios esperan obtener?

Cumplir con los tiempos establecidos de entrega del sistema, detectando los problemas que puedan surgir durante el desarrollo de la plataforma y mediante el uso del kanban tener el control visual de los estados de cada tarea y el seguimiento del proyecto.

3. ORGANIZACIÓN DEL EQUIPO

Integrante	Rol	Responsabilidades principales
Gustavo Inurreta Brito	Líder del Proyecto	Coordinación del equipo, seguimiento de actividades, desarrollo backend, frontend y documentación
Michael Román Alavez Collí	Desarrollador Backend	Desarrollo backend, lógica de negocio y administración de base de datos
Carlos Alberto Castillo Pinzon	Desarrollador Full Stack y Tester	Desarrollo backend, frontend, pruebas funcionales y corrección de errores
Rodrigo Alejandro Barbosa Vidal	Diseñador y Tester	Desarrollo de interfaces, apoyo backend y documentación

4. DESCRIPCIÓN DE LOS ROLES

Líder del Proyecto (Gustavo Inurreta Brito)

- Coordinar las actividades del equipo.
- Supervisar el cumplimiento de los sprints.
- Dar seguimiento al avance general.
- Participar en el desarrollo backend y frontend.
- Colaborar en la documentación del proyecto.

Desarrollador Backend (Michael Román Alavez Collí)

- Diseñar y administrar la base de datos.
- Desarrollar la lógica de negocio.
- Implementar funcionalidades del servidor.
- Garantizar la integridad de los datos.

Desarrollador Full Stack y Tester (Carlos Alberto Castillo Pinzon)

- Desarrollar módulos frontend y backend.
- Integrar componentes del sistema.
- Realizar pruebas funcionales.
- Detectar y corregir errores.

Desarrollador Frontend (Rodrigo Alejandro Barbosa Vidal)

- Diseñar interfaces de usuario.
- Implementar formularios y vistas.
- Mejorar la experiencia del usuario.
- Colaborar en la documentación del sistema.

5. PLANIFICACIÓN DEL DESARROLLO

Organización del trabajo

Las actividades serán distribuidas de acuerdo con los roles y conocimientos de cada integrante.

Asignación de tareas

Las tareas se asignarán durante las reuniones de planificación de sprint, considerando la complejidad de cada actividad y la disponibilidad de los integrantes.

Seguimiento del avance

Se utilizará un tablero Kanban para monitorear el estado de las tareas.

Gestión de cambios e incidencias

Los cambios en los requerimientos serán analizados por el equipo antes de su implementación para evaluar impacto y prioridad.

Comunicación del equipo

La comunicación se realizará mediante el uso de las herramientas de mensajería de WhatsApp.

6. PLAN DE SPRINTS O ITERACIONES

Sprint 1 – Planeación y estructura inicial

Objetivo

Preparar la base del sistema y la estructura de la base de datos.

Actividades

Configuración del entorno, levantamiento de Laravel, conexión al control de versiones GitHub, desarrollo de interfaces de usuario y levantamiento del servidor local con la herramienta de pruebas XAMPP.

Entregables

Base de datos relacional, inicio de sesión funcional, así como el prototipo de la vista de administrador.

Sprint 2 – Gestión de bienes

Objetivo

Desarrollar el módulo principal de inventario.

Actividades

Desarrollo de los módulos de administrador funcional, como registro, modificación y baja de bienes, así como la generación de códigos de barras al crear un nuevo bien.

Entregables

Módulo de bienes funcional y generación automática de códigos de barras.

Sprint 3 – Gestión de personal y áreas

Objetivo

Implementar la administración de personal y áreas.

Actividades

Desarrollo de los módulos de registro de personal y áreas, así como la baja de los mismos.

Entregables

Módulo de personal y áreas funcionales dentro de la vista de administrador.

Sprint 4 – Asignaciones y consultas

Objetivo

Implementar la asignación y consulta de bienes.

Actividades

Asignación y reasignación de bienes, así como la consulta mediante código de barras y filtros de búsqueda de bienes.

Entregables

Sistema de asignación de bienes funcional y consulta por código de barras dentro de la vista de administración.

Sprint 5 – Vista de Visualizador

Implementar la vista y funcionalidades del usuario visualizador.

Actividades

Desarrollo de la vista de visualizador para la consulta de bienes, personal y áreas. Implementación de filtros de búsqueda y visualización de reportes sin permisos de edición o eliminación de información.

Entregables

Vista de visualizador funcional con acceso a consultas, filtros y reportes.

Sprint 6 – Reportes y pruebas finales

Objetivo

Generar reportes y realizar validaciones finales.

Actividades

Desarrollare que se pueda generar reportes, hacer pruebas y corregir los errores

Entregables

Sistema completo funcional con reportes operativos.

7. HERRAMIENTAS DE GESTIÓN Y COLABORACIÓN

Herramienta	Uso
-------------	-----

GitHub	Control de versiones y almacenamiento del repositorio del proyecto.
Notion	Organización de tareas, documentación y seguimiento de actividades.
Google Docs	Elaboración y colaboración en la documentación del proyecto.
Visual Studio Code	Desarrollo y edición del código fuente del sistema.
MySQL	Gestión y almacenamiento de la base de datos.
WhatsApp	Comunicación y coordinación constante entre los integrantes del equipo de desarrollo y el cliente.

GitHub

Almacenar el código fuente del proyecto y llevar un control de versiones que permita el trabajo colaborativo entre los integrantes.

Notion

Utilizado para organizar las actividades, asignar tareas y dar seguimiento al avance de cada sprint.

Google Docs

Permitir la elaboración colaborativa de la documentación del proyecto, facilitando que todos los integrantes puedan realizar modificaciones y aportar información en tiempo real.

Visual Studio Code

El entorno de desarrollo principal para la implementación del sistema utilizando Laravel, PHP, HTML, CSS y JavaScript.

MySQL

Utilizado para almacenar y administrar la información del inventario, personal, áreas y asignaciones de bienes.

WhatsApp

La herramienta principal de comunicación del equipo, permitiendo coordinar actividades, compartir avances y resolver dudas durante el desarrollo del proyecto.

8. ESTRATEGIA DE CONTROL DE VERSIONES

Herramienta utilizada

Para el control de versiones se utilizará GitHub, permitiendo gestionar los cambios realizados por cada integrante y mantener un historial de las modificaciones efectuadas durante el desarrollo del proyecto.

Repositorio del proyecto

El repositorio del proyecto estará organizado en seis ramas principales:

Gustavo Inurreta Brito: rama de trabajo individual destinada al desarrollo y gestión de las actividades asignadas a este integrante.

Carlos Alberto Castillo Pinzon: rama de trabajo individual destinada al desarrollo y gestión de las actividades asignadas a este integrante.

Rodrigo Alejandro Barbosa Vidal: rama de trabajo individual destinada al desarrollo y gestión de las actividades asignadas a este integrante.

Michael Román Alavez Collí: rama de trabajo individual destinada al desarrollo y gestión de las actividades asignadas a este integrante.

Pruebas: rama utilizada para integrar, revisar y validar los cambios realizados por los integrantes antes de incorporarlos a la versión final del sistema.

Main: rama principal que contendrá la versión estable, validada y completa del sistema.

Estrategia de ramas

Cada integrante trabajará en su rama correspondiente, donde realizará las tareas y modificaciones que le hayan sido asignadas. Una vez finalizadas, los cambios serán enviados a la rama Pruebas para su revisión y validación.

Después de verificar que las funcionalidades operan correctamente y no generan conflictos con otros módulos, los cambios serán incorporados a la rama Main, que almacenará la versión estable del sistema.

Procedimiento para la integración de cambios

1. Cada integrante clona el repositorio del proyecto.
2. Realiza las actividades asignadas en su rama correspondiente.

3. Registra los cambios realizados.
4. Una vez finalizada la tarea, los cambios se integran en la rama Pruebas.
5. El equipo revisa y valida el funcionamiento de las nuevas funcionalidades.
6. Si no se detectan errores, los cambios se integran en la rama Main.
7. La rama Main mantiene siempre la versión estable y actualizada del sistema.

9. GESTIÓN DE RIESGOS

Riesgo	Impacto	Acción Preventiva
Retrasos en el desarrollo de actividades	Alto	Realizar seguimiento semanal de tareas y establecer fechas límite para cada sprint.
Cambios en los requerimientos del sistema	Medio	Mantener comunicación constante con el cliente y validar los requerimientos antes de iniciar cada sprint.
Conflictos en la integración del código	Alto	Utilizar ramas individuales y realizar pruebas antes de integrar cambios a la rama principal.
Errores en la base de datos	Alto	Diseñar y validar la estructura de la base de datos desde las primeras etapas del proyecto.
Pérdida de información del proyecto	Alto	Mantener respaldos periódicos del código y de la base de datos utilizando GitHub y copias de seguridad locales.
Fallas en el servidor local de desarrollo	Medio	Verificar periódicamente la configuración del entorno de desarrollo y mantener respaldos del proyecto.
Ausencia o falta de disponibilidad de algún integrante	Medio	Distribuir adecuadamente las tareas y documentar los avances para facilitar la continuidad del trabajo.
Errores detectados en etapas finales del proyecto	Alto	Realizar pruebas continuas durante cada sprint para identificar problemas de manera temprana.
Problemas en la generación de reportes o consultas	Medio	Validar los datos y realizar pruebas funcionales antes de cada entrega.
Falta de coordinación entre los integrantes	Medio	Mantener comunicación constante mediante WhatsApp y reuniones de seguimiento para revisar avances y resolver dudas.

Estrategia de mitigación

Para reducir el impacto de los riesgos identificados, el equipo llevará un seguimiento continuo del avance del proyecto mediante reuniones periódicas y el uso de herramientas de colaboración. Las pruebas funcionales se realizan durante cada sprint para detectar y corregir errores de manera temprana, evitando problemas mayores en las etapas finales del desarrollo.

1. Reglas de Clean Code del Equipo

Reglas de Clean Code del Equipo

Sistema de Inventario Escolar

Regla 1. Utilizar nombres claros y descriptivos

Todas las variables, métodos, clases y archivos deberán tener nombres que describan claramente su propósito.

Se evitarán abreviaciones poco entendibles o nombres genéricos que dificulten la comprensión del código.

Ejemplos:

Correcto:

- BienController
- ReportesController
- AsignacionController
- nombre_bien
- codigo_barras
- fecha_registro

Incorrecto:

- bc
- dato
- x
- variable1
- prueba

Esta regla facilita la lectura del código y permite que cualquier integrante del equipo comprenda rápidamente la funcionalidad desarrollada.

Regla 2. Separar responsabilidades por módulo

Cada controlador deberá encargarse únicamente de las funciones relacionadas con su módulo.

Ejemplo:

- BienController administrará los bienes.
- AreaController administrará las áreas.
- PersonalController administrará el personal.
- AsignacionController administrará las asignaciones.
- ReportesController administrará los reportes.

Esta regla evita controladores demasiado grandes y facilita el mantenimiento del sistema.

Regla 3. Validar todos los datos antes de almacenarlos

Antes de guardar cualquier información en la base de datos deberán realizarse las validaciones correspondientes.

Las validaciones permitirán evitar:

- Campos obligatorios vacíos.
- Información duplicada.
- Datos con formato incorrecto.
- Relaciones inválidas.
- Errores en la base de datos.

Para ello se utilizarán las validaciones propias de Laravel mediante Request Validation.

Regla 4. Evitar la duplicación de código

No se permitirá copiar y pegar la misma lógica en diferentes archivos.

Cuando una funcionalidad sea utilizada en varios módulos deberá implementarse mediante funciones reutilizables, servicios o métodos comunes.

Ejemplos:

- Generación de números de inventario.
- Generación de códigos de barras.
- Exportación de reportes.
- Validaciones compartidas.

Esta regla facilita el mantenimiento del sistema y reduce el riesgo de inconsistencias.

Regla 5. Mantener funciones pequeñas y enfocadas

Cada función deberá realizar una sola tarea específica.

No deberán desarrollarse métodos demasiado largos que mezclen diferentes responsabilidades.

Por ejemplo, un método encargado de exportar reportes no deberá realizar consultas, validaciones, cálculos y generación de archivos al mismo tiempo.

Cada proceso deberá separarse en métodos independientes.

Regla 6. Utilizar comentarios únicamente cuando sean necesarios

Los comentarios solamente deberán utilizarse cuando sea necesario explicar procesos complejos.

No se comentará código cuya función sea evidente.

Los comentarios deberán mantenerse actualizados para evitar información incorrecta.

Se utilizarán principalmente en:

- Procesos de importación.
- Exportaciones.
- Generación de códigos de barras.
- Algoritmos complejos.

Regla 7. Utilizar constantes para valores fijos

Los estados, roles y demás valores constantes no deberán escribirse directamente dentro del código.

Se utilizarán constantes o enumeraciones.

Ejemplo:

- Estado Activo
- Estado Inactivo
- Administrador
- Visualizador

Esto facilitará futuras modificaciones y mejorará la legibilidad del código.

Regla 8. Mantener un formato uniforme del código

Todo el equipo deberá respetar el mismo estilo de programación.

Se utilizarán las convenciones de Laravel y PHP respecto a:

- Indentación.
- Organización de métodos.
- Espacios.
- Llaves.
- Orden de clases.
- Organización de carpetas.

De esta manera el código mantendrá una apariencia uniforme durante todo el proyecto.

Compromiso del equipo

El equipo se compromete a respetar estas reglas durante todo el desarrollo del Sistema de Inventario Escolar.

La aplicación de estas prácticas permitirá desarrollar un software más limpio, mantenible y fácil de ampliar en futuras versiones.

2. Definition of Done del equipo

Definition of Done (DoD)

Sistema de Inventario Escolar

Criterios de la Definition of Done

Una funcionalidad del Sistema de Inventario Escolar solamente será considerada terminada cuando cumpla con todos los siguientes criterios:

Desarrollo

1. Cumple completamente con el requerimiento solicitado por el cliente o el equipo.
2. Fue desarrollada dentro del módulo correspondiente.
3. Sigue las reglas de Clean Code establecidas por el equipo.
4. Utiliza nombres claros y descriptivos para clases, métodos y variables.
5. No contiene código duplicado.
6. No contiene código innecesario o comentado.
7. Mantiene una estructura organizada y fácil de mantener.

Validaciones

8. Todos los datos ingresados son validados correctamente.
9. La información se almacena correctamente en la base de datos.
10. Las relaciones entre bienes, áreas, personal y usuarios funcionan correctamente.
11. Se respetan los permisos definidos para cada rol del sistema.
12. La funcionalidad no genera errores visibles.
13. Los mensajes mostrados al usuario son claros.

Pruebas

14. La funcionalidad fue probada manualmente.
15. Se realizaron pruebas funcionales cuando fue necesario.
16. No afecta el funcionamiento de otros módulos.
17. Los reportes generan información correcta.
18. Las exportaciones en CSV, Excel y PDF funcionan correctamente.

Calidad

19. El diseño mantiene la apariencia del resto del sistema.
20. La funcionalidad cumple con los estándares de seguridad establecidos.
21. El historial registra correctamente las acciones cuando corresponde.
22. La funcionalidad fue probada en diferentes escenarios de uso.

Integración

- 23. El código fue revisado por otro integrante del equipo.
- 24. Fue integrado correctamente al repositorio GitHub.
- 25. No existen conflictos pendientes antes de integrar los cambios.
- 26. La documentación fue actualizada cuando fue necesario.
- 27. La funcionalidad se encuentra lista para ser presentada al cliente.

1. Generalidades del Proyecto

Objetivo General

Desarrollar un sistema web que permita administrar el inventario de bienes escolares, registrando artículos, áreas, responsables, asignaciones, movimientos, pendientes y reportes, con el fin de mantener un control ordenado, consultable y actualizado de los recursos de la institución.

Problema que Resuelve

En una institución educativa, el control manual de bienes puede generar pérdida de información, duplicidad de registros, falta de seguimiento sobre responsables, dificultad para ubicar artículos y poca claridad sobre el estado actual del inventario. El sistema resuelve este problema centralizando la información en una aplicación web, permitiendo registrar bienes, asignarlos a personal o áreas, consultar su historial, generar reportes y revisar artículos pendientes o en mantenimiento.

Tecnologías Utilizadas

- **Backend:** PHP 8.2 con Laravel 12.
- **Frontend:** Blade, HTML, CSS y JavaScript.
- **Base de Datos:** MySQL / MariaDB mediante migraciones de Laravel.
- **ORM:** Eloquent.
- **Autenticación:** Sistema nativo de Laravel con roles de usuario.
- **Reportes:** Exportación en CSV, Excel (XLSX) y PDF.
- **Códigos:** Generación de códigos de barras y códigos QR mediante librerías de PHP.
- **Servidor Local:** XAMPP.

Arquitectura General del Sistema

El proyecto utiliza una arquitectura **MVC (Modelo-Vista-Controlador)** propia de Laravel:

- **Modelo:** Representa las entidades principales del sistema, como bienes, áreas, personal, usuarios, marcas e historial de asignaciones.
- **Vista:** Contiene las pantallas administrativas y públicas desarrolladas con Blade.
- **Controlador:** Procesa solicitudes, valida datos, ejecuta reglas de negocio y comunica los modelos con las vistas.
- **Rutas:** Definen los accesos del sistema, tanto públicos como protegidos por autenticación.
- **Base de Datos:** Almacena la información persistente del inventario y sus relaciones.

Módulos Principales

- **Autenticación:** Inicio y cierre de sesión.
- **Dashboard:** Vista principal del sistema después de iniciar sesión.
- **Gestión de Bienes:** Registro, edición, búsqueda, importación, papelera, restauración y eliminación de bienes.
- **Áreas:** Administración de áreas escolares.
- **Personal:** Administración del personal responsable de bienes.
- **Asignaciones:** Asignación, transferencia, devolución y reasignación de bienes.
- **Historial:** Consulta y exportación de movimientos realizados.
- **Reportes:** Generación de reportes filtrados del inventario.
- **Pendientes:** Seguimiento de bienes sin asignar, sin área, dañados, en revisión o en mantenimiento.
- **Usuarios:** Administración de usuarios con rol administrador o visualizador.
- **Consulta Pública por QR:** Acceso público al detalle de un bien mediante código de barras o QR.

2. Estrategia de Pruebas

Tipos de Pruebas a Realizar

- **Pruebas Unitarias:** Verificarán componentes individuales, como validaciones, generación de códigos, filtros, clasificación de pendientes y preparación de datos para reportes.
- **Pruebas de Integración:** Revisarán la comunicación entre módulos, por ejemplo bienes con áreas, personal, historial, reportes y base de datos.
- **Pruebas End-to-End (E2E):** Simularán recorridos completos desde la perspectiva del usuario, como iniciar sesión, registrar un bien, asignarlo, generar reporte y cerrar sesión.
- **Pruebas de Seguridad y Permisos:** Validarán que solo usuarios administradores puedan crear, modificar o eliminar información crítica.
- **Pruebas de Validación de Datos:** Confirmarán que los formularios rechacen información incompleta, inválida o fuera de catálogo.
- **Pruebas de Exportación:** Comprobarán que los archivos CSV, Excel y PDF se generen con información correcta y filtros aplicados.

Momento de Ejecución

1. **Durante el desarrollo:** Pruebas unitarias para validar reglas de negocio conforme se implementen.
2. **Al terminar cada módulo:** Pruebas de integración entre pantalla, controlador, modelo y base de datos.
3. **Antes de una entrega parcial:** Pruebas E2E de los flujos principales del sistema.
4. **Antes de la entrega final:** Ejecución completa de pruebas funcionales, integración, permisos, reportes y E2E.
5. **Después de corregir errores:** Pruebas de regresión para confirmar que los cambios no afectaron funciones ya existentes.

Responsables

- Rodrigo Alejandro Barbosa Vidal

Herramientas para Documentar Resultados

- **Capturas de pantalla:** Evidencia visual de la ejecución del sistema y de las pruebas realizadas.
- **Bloc de notas:** Archivo de texto con las observaciones y anotaciones generadas durante las pruebas.
- **Documento de Google o Microsoft Word:** Documento que contenga la descripción de los fallos encontrados, acompañado de las capturas de pantalla correspondientes como evidencia de cada incidencia.

Justificación de la Estrategia

La estrategia de pruebas se basa en la ejecución de casos de prueba y en la documentación de las evidencias obtenidas durante el proceso. Para cada prueba realizada se recopilarán **capturas de pantalla** como evidencia visual del funcionamiento del sistema, además de un **bloc de notas** para registrar observaciones y un **documento de Google o Microsoft Word** donde se describan los fallos detectados, acompañados de sus respectivas capturas de pantalla.

Esta metodología facilita la identificación, seguimiento y corrección de errores, ya que cada incidencia queda debidamente documentada y respaldada con evidencia visual, permitiendo verificar que las correcciones implementadas resuelvan los problemas detectados y asegurando un adecuado control de calidad del sistema.

3. Plan de Pruebas Unitarias

PU-01: Autenticación

Campo	Descripción
Identificador	PU-01
Módulo	Autenticación
Función/Método	AuthController::login()
Objetivo	Verificar que el sistema permita iniciar sesión solo con credenciales válidas.
Datos de Entrada	• Usuario: admin@escuela.com con contraseña correcta. • Usuario inválido o contraseña incorrecta.
Resultado Esperado	Con credenciales correctas redirige al dashboard. Con credenciales inválidas muestra mensaje de error y deniega el acceso.
Resultado Obtenido	Pendiente
Estado	Pendiente

PU-02: Registro de Bienes

Campo	Descripción
Identificador	PU-02
Módulo	Bienes
Función/Método	BienController::store()
Objetivo	Verificar que un bien se registre correctamente con datos válidos.
Datos de Entrada	Nombre del bien, marca, modelo, serie, valor, área, responsable y estatus válido.
Resultado Esperado	El bien se guarda en la base de datos, se genera un número de inventario y código de barras de forma automática (si no fueron capturados) y retorna un mensaje de éxito.
Resultado Obtenido	Pendiente
Estado	Pendiente

PU-03: Importación Masiva

Campo	Descripción
Identificador	PU-03
Módulo	Bienes
Función/Método	BienController::importExcel()
Objetivo	Verificar que el sistema procese correctamente un archivo CSV, XLS o XLSX con bienes.
Datos de Entrada	Archivo con encabezados obligatorios: nombre_bien, marca, modelo, serie, codigo_barras, id_area, id_personal, estatus.
Resultado Esperado	Los bienes válidos se importan, se vinculan a las áreas/personal correspondientes, se evitan duplicados y se genera un reporte de errores por fila si existen fallas.
Resultado Obtenido	Pendiente
Estado	Pendiente

PU-04: Gestión de Asignaciones

Campo	Descripción
Identificador	PU-04
Módulo	Asignaciones
Función/Método	AsignacionController::store()
Objetivo	Verificar que una asignación registre el movimiento y actualice el estado del bien.
Datos de Entrada	ID del bien, ID del responsable, ID del área, tipo de movimiento (Asignacion) y observaciones.
Resultado Esperado	Se crea un registro en la tabla de historial, el bien queda relacionado con el nuevo responsable o área y su estatus cambia a Asignado.
Resultado Obtenido	Pendiente
Estado	Pendiente

PU-05: Resolución de Pendientes

Campo	Descripción
Identificador	PU-05
Módulo	Pendientes
Función/Método	PendientesController::resolver()
Objetivo	Verificar que un pendiente pueda resolverse correctamente cambiando el flujo del bien.
Datos de Entrada	Acción: Asignar; nuevo_estatus: Resuelto; notas: Equipo asignado al laboratorio.
Resultado Esperado	El estatus del bien cambia a Asignado o Disponible según corresponda, y se añade un movimiento de tipo Resolución en la bitácora.
Resultado Obtenido	Pendiente
Estado	Pendiente

PU-06: Exportación de Reportes

Campo	Descripción
Identificador	PU-06
Módulo	Reportes
Función/Método	ReportesController::export()
Objetivo	Verificar que el sistema exporte reportes en formatos válidos respetando los filtros.
Datos de Entrada	Formato: csv, xlsx o pdf; filtros: área, responsable, estatus y rango de fechas.
Resultado Esperado	Descarga inmediata de un archivo legible con los registros filtrados. Si el formato es inválido, aborta con un mensaje de error controlado.
Resultado Obtenido	Pendiente
Estado	Pendiente

PU-07: Consulta Pública

Campo	Descripción
Identificador	PU-07
Módulo	Consulta Pública
Función/Método	BienController::detallePublico()
Objetivo	Verificar que un código de barras o número de inventario renderice el detalle público del bien.
Datos de Entrada	• Código existente: ABC123 • Código inexistente: NO EXISTE
Resultado Esperado	Si existe, muestra la ficha técnica pública del bien. Si no existe, redirecciona a una vista 404 controlada o despliega un mensaje de error.
Resultado Obtenido	Pendiente
Estado	Pendiente

4. Plan de Pruebas de Integración

PI-01: Acceso Seguro al Dashboard

- **Módulos Involucrados:** Autenticación, Usuarios, Dashboard.
- **Objetivo:** Confirmar que un usuario autenticado acceda correctamente al panel principal y sus módulos correspondientes.
- **Flujo Esperado:** El usuario abre el login, captura credenciales válidas, el sistema valida, inicia la sesión en el servidor y lo redirige al Dashboard.
- **Resultado Esperado:** El Dashboard renderiza los componentes interactivos únicamente si el token de sesión está activo.
- **Estado:** Pendiente

PI-02: Integración de Catálogos en Bienes

- **Módulos Involucrados:** Bienes, Áreas, Personal, Marcas, Base de Datos.
- **Objetivo:** Verificar que el registro de un bien se integre de manera íntegra con catálogos relacionados (llaves foráneas).
- **Flujo Esperado:** El administrador registra un bien seleccionando un área, un responsable y una marca existente desde los desplegables de la interfaz.
- **Resultado Esperado:** El bien se inserta correctamente y se refleja en los listados mostrando los nombres reales de las relaciones (no solo los IDs).
- **Estado:** Pendiente

PI-03: Trazabilidad de Asignaciones

- **Módulos Involucrados:** Asignaciones, Bienes, Personal, Áreas, Historial.
- **Objetivo:** Comprobar que una asignación actualice el estado del bien y genere trazabilidad inmediata.
- **Flujo Esperado:** Se selecciona un bien disponible, se asigna a un responsable y área, se procesa la transacción y se procede a auditar en el módulo de Historial.

- **Resultado Esperado:** El bien se actualiza a estatus Asignado y el historial inserta un registro detallando los datos anteriores y los nuevos.
- **Estado:**Pendiente

PI-04: Consistencia en Importación Masiva

- **Módulos Involucrados:** Bienes, Importación, Áreas, Personal, Historial.
- **Objetivo:** Validar que la importación masiva cree bienes respetando y mapeando correctamente las relaciones necesarias.
- **Flujo Esperado:** El administrador carga un archivo CSV/XLSX. El sistema normaliza los datos, evalúa si las áreas o personal existen (o los crea si la lógica de negocio lo define) y almacena los bienes.
- **Resultado Esperado:** Los bienes válidos ingresan al inventario y las filas con errores se notifican sin romper ni revertir la importación de las filas correctas.
- **Estado:**Pendiente

PI-05: Integridad de Datos Filtrados en Reportes

- **Módulos Involucrados:** Inventario, Reportes, Filtros, Exportación.
- **Objetivo:** Confirmar que los reportes generados utilicen exactamente los mismos filtros aplicados en la pantalla de previsualización.
- **Flujo Esperado:** El usuario filtra la tabla de inventario por un área específica y un estatus. Posteriormente da clic en el botón de exportación.
- **Resultado Esperado:** El documento generado (Excel/PDF) contiene única y exclusivamente los registros que se visualizaban bajo esos filtros.
- **Estado:**Pendiente

PI-06: Generación y Lectura de QR

- **Módulos Involucrados:** Bienes, QR / Código de Barras, Consulta Pública.
- **Objetivo:** Verificar que el código generado dinámicamente en el backend apunte de manera correcta al endpoint de consulta pública.
- **Flujo Esperado:** Al registrar un bien, el sistema renderiza el código QR. Se extrae la URL embebida en dicho código y se procesa mediante una petición GET sin sesión.
- **Resultado Esperado:** La ruta responde con código 200 OK y muestra los datos generales del bien escolar consultado.
- **Estado:** Pendiente

Justificación de Integraciones Críticas: Estas integraciones son críticas porque el sistema no almacena bienes aislados; depende enteramente de la cohesión entre inventario, responsables, áreas y movimientos. Si una asignación no genera historial, se pierde la trazabilidad legal del bien. Si un reporte ignora los filtros, se entrega información errónea a la dirección de la escuela. Por ende, estas pruebas blindan los puntos donde los módulos coexisten.

5. Plan de Pruebas End-to-End (E2E)

E2E-01: Flujo Completo de Registro de un Bien

- **Actor Principal:** Administrador.
- **Precondiciones:** Cuenta activa del administrador; áreas y personal dados de alta previamente en el sistema.
- **Pasos a Seguir:**
 1. Abrir el navegador e ingresar a la URL del sistema.
 2. Iniciar sesión con las credenciales de administrador.
 3. Navegar al módulo de **Bienes** mediante el menú lateral.

4. Hacer clic en "Registrar nuevo bien" y rellenar el formulario con datos válidos.
 5. Asignar el área, responsable, marca correspondientes y fijar estatus como "Disponible".
 6. Guardar el formulario.
 7. Buscar el bien en la tabla general para verificar su inserción.
 8. Cerrar sesión.
- **Resultado Esperado:** El bien se crea sin contratiempos, su fila aparece en el listado de inventario y la sesión se destruye de forma segura al finalizar.
 - **Estado:** Pendiente

E2E-02: Ciclo de Asignación y Auditoría

- **Actor Principal:** Administrador.
- **Precondiciones:** Contar con al menos un bien disponible y personal activo en el sistema.
- **Pasos a Seguir:**
 1. Iniciar sesión.
 2. Entrar al módulo de **Asignaciones**.
 3. Seleccionar el bien escolar mediante el buscador o selector.
 4. Elegir el responsable y el área destino.
 5. Establecer tipo de movimiento como Asignación y redactar una observación.
 6. Guardar la transacción.
 7. Dirigirse al módulo de **Historial**.
 8. Buscar el bien y comprobar la línea de tiempo del movimiento.
 9. Cerrar sesión.
- **Resultado Esperado:** El estatus del bien muta a Asignado, se restringe su asignación doble y la bitácora refleja detalladamente quién lo tiene ahora bajo su resguardo.
- **Estado:** Pendiente.

E2E-03: Auditoría y Exportación de Reporte Anual

- **Actor Principal:** Administrador o Visualizador.
- **Precondiciones:** Existencia de registros cargados en múltiples áreas de la institución.
- **Pasos a Seguir:**
 1. Iniciar sesión.
 2. Acceder al módulo de **Reportes**.
 3. Aplicar un filtro selectivo (ej. Área: Laboratorio de Cómputo, Estatus: En Mantenimiento).
 4. Validar visualmente los resultados arrojados en la tabla dinámica.
 5. Hacer clic en "Exportar a PDF" o "Exportar a Excel".
 6. Descargar y abrir el archivo localmente para inspeccionar los datos.
 7. Cerrar sesión.
- **Resultado Esperado:** El archivo descargado emula con exactitud la consulta en pantalla, con un diseño limpio, columnas alineadas y sin romper celdas.
- **Estado:** Pendiente

E2E-04: Carga de Inventario por Lote (Migración)

- **Actor Principal:** Administrador.
- **Precondiciones:** El usuario dispone de una plantilla oficial en formato .xlsx con información mixta de bienes (correctos y con celdas vacías a propósito).
- **Pasos a Seguir:**
 1. Iniciar sesión.
 2. Entrar a la sección de **Bienes** e ir a la opción "Importar masivamente".
 3. Arrastrar o seleccionar el archivo .xlsx de prueba.
 4. Hacer clic en "Procesar Importación".
 5. Verificar los mensajes informativos en pantalla (filas exitosas vs filas rechazadas).
 6. Buscar en el inventario uno de los bienes importados exitosamente.

7. Cerrar sesión.

- **Resultado Esperado:** El sistema discrimina los registros corruptos notificando la fila exacta, e inserta fluidamente los registros válidos.
- **Estado:** Pendiente

E2E-05: Escaneo y Consulta Externa (QR)

- **Actor Principal:** Usuario Externo / Personal de la institución.
- **Precondiciones:** Haber impreso o disponer en pantalla del QR generado para un bien específico.
- **Pasos a Seguir:**
 1. Escanear el código QR mediante un dispositivo móvil o ingresar de forma directa al enlace público (/consulta-bien/{código}).
 2. El sistema procesa el parámetro de la URL.
 3. Buscar el código en la base de datos de manera interna.
 4. Renderizar la pantalla pública de cara al usuario.
- **Resultado Esperado:** El usuario visualiza la ficha de información pública del bien (nombre, estatus, área) sin necesidad de pasar por una pantalla de login o requerir credenciales del sistema.
- **Estado:** Pendiente

6. Matriz de Cobertura de Pruebas

Requerimiento Funcional (RF)	Descripción del Requerimiento	Prueba Unitaria	Integración	End-to-End (E2E)
RF-01	El sistema debe permitir iniciar y cerrar sesión.	PU-01	PI-01	E2E-01, E2E-02, E2E-03

RF-02	El sistema debe permitir registrar, editar, buscar y eliminar bienes.	PU-02	PI-02	E2E-01
RF-03	El sistema debe generar número de inventario y código de barras/QR para bienes.	PU-02, PU-07	PI-06	E2E-05
RF-04	El sistema debe permitir importar bienes desde archivos CSV, XLS o XLSX.	PU-03	PI-04	E2E-04
RF-05	El sistema debe administrar áreas escolares.	PU-02	PI-02, PI-03	E2E-01, E2E-02
RF-06	El sistema debe administrar personal responsable.	PU-04	PI-02, PI-03	E2E-02
RF-07	El sistema debe permitir asignar,	PU-04	PI-03	E2E-02

	transferir o devolver bienes.			
RF-08	El sistema debe registrar historial de movimientos.	PU-04, PU-05	PI-03	E2E-02
RF-09	El sistema debe identificar y resolver bienes pendientes.	PU-05	PI-03	E2E-02
RF-10	El sistema debe generar reportes filtrados y exportables.	PU-06	PI-05	E2E-03
RF-11	El sistema debe permitir consulta pública de bienes por código.	PU-07	PI-06	E2E-05
RF-12	El sistema debe controlar permisos por rol administrador o visualizador.	PU-01	PI-01, PI-02	E2E-01, E2E-03

7. Criterios de Aceptación

Una funcionalidad o módulo se dará formalmente por **Aceptado** cuando cumpla rigurosamente con los siguientes puntos:

1. **Funcionamiento Correcto:** Realiza la acción nativa para la cual fue diseñado (crear, leer, actualizar, borrar) sin arrojar excepciones de código o pantallas caídas.
2. **Cumplimiento del Alcance:** El resultado final se alinea con lo estipulado en la documentación inicial del proyecto o SRS.
3. **Validación de Datos en Formularios:** El sistema reacciona deteniendo las solicitudes ante campos obligatorios vacíos o tipos de datos erróneos, mostrando mensajes amigables al usuario (diseño *Frontend* accesible).
4. **Manejo Controlado de Errores:** En caso de fallas críticas (ej. caída de base de datos), el sistema muestra pantallas de error controladas evitando exponer líneas de código sensibles o comprometer información previa.
5. **Integración de Relaciones:** Los cambios se ven reflejados instantáneamente en cascada en los módulos adyacentes (Historial, Reportes, Contadores del Dashboard).
6. **Persistencia Comprobada:** Los datos se registran o modifican de manera real y permanente en las tablas correspondientes de MySQL/MariaDB.
7. **Control estricto de Permisos (RBAC):** Las rutas protegidas impiden el paso mediante URL directas a usuarios con el rol Visualizador u operadores no autenticados.
8. **Trazabilidad Obligatoria:** Todo cambio físico de un bien (reasignaciones, bajas o mantenimiento) genera de forma automática su fila correspondiente en la bitácora histórica.
9. **Fidelidad de Reportes:** Los archivos exportados son legibles, están limpios de caracteres extraños y respetan las restricciones de los filtros aplicados.
10. **Cero Bloqueos Críticos:** No persisten errores de severidad alta que impidan el flujo normal de las operaciones diarias del inventario (ej. fallas al iniciar sesión).

8. Riesgos y Limitaciones

Riesgo / Limitación	Impacto	Acción Mitigatoria / Solución
Falta de datos de prueba suficientes	Medio	Diseñar scripts de inserción masiva (Seeders y Factories en Laravel) con datos ficticios realistas para estresar filtros y reportes.
Cambios frecuentes en requerimientos	Alto	Congelar el alcance funcional de la entrega y actualizar la matriz de cobertura ante variaciones autorizadas por el evaluador.
Formatos erróneos en archivos de importación	Alto	Forzar la descarga de una plantilla oficial (.xlsx) en el sistema y validar de forma estricta los nombres de las cabeceras en el backend antes de iterar las filas.
Inconsistencias en relaciones de la BD	Alto	Configurar de forma correcta restricciones FOREIGN KEY a nivel de base de datos y usar operaciones de Eloquent controladas en transacciones (DB::transaction).
Suplantación o fallas en permisos de rol	Alto	Implementar un desarrollo riguroso de Middlewares de Laravel en las rutas del

		backend, no limitándose solo a esconder botones en la vista Blade.
Tiempo limitado para pruebas manuales	Medio	Priorizar y asegurar al 100% el funcionamiento del flujo central: Inicio de sesión → Gestión de Bienes → Asignaciones → Historial.
Dependencia de extensiones y librerías externas	Medio	Asegurar la instalación previa de las extensiones gd, zip y dom en el archivo php.ini de XAMPP para evitar errores al procesar QR, Excel o PDF.
Registros duplicados durante la importación	Medio	Configurar validaciones que contrasten números de serie o códigos de barras existentes (unique en la base de datos) antes de confirmar las inserciones.
Inaccesibilidad del QR en redes locales	Bajo	Configurar la variable APP_URL en el archivo .env de Laravel con la dirección IP local de la computadora del servidor en lugar de usar localhost.

Conclusión

En conclusión, la planeación del **Sistema de Gestión de Inventario y Resguardo de Bienes Institucionales** nos permitió tener una idea más clara de lo que queremos desarrollar y de los problemas que buscamos solucionar. El sistema está pensado para facilitar el control de los bienes, mejorar la organización de la información y hacer más sencillo saber qué bienes existen, dónde se encuentran y quién los tiene asignados.

Durante la realización del documento también pudimos entender la importancia de organizar el trabajo antes de comenzar con el desarrollo. Definir las funciones, los usuarios, las reglas y las actividades nos ayuda a evitar confusiones y a saber qué debe hacer cada integrante del equipo. La forma de trabajo que elegimos también nos permitirá llevar un mejor control de las tareas y de los avances del proyecto.

Finalmente, consideramos que toda la información reunida en este documento será de gran ayuda cuando comencemos a desarrollar el sistema, ya que podremos utilizarla como una guía para saber qué debemos realizar y cómo debe funcionar cada parte. También nos ayudará a detectar errores y comprobar que el sistema cumpla con lo que se planteó desde el inicio. Con este proyecto esperamos lograr una herramienta que haga más fácil y ordenado el manejo del inventario de la institución.